Test Cases Documentation: Roman Numeral Converter

Author: Juan Iznardo (alu142910) **Date:** April 2026 **Subject:** Functional Test Cases for Integer ↔ Roman Numeral Conversion Functions

---

Table of Contents

1. Overview
2. Function Analysis
3. Set 1: Manual Test Cases
4. Set 2: AI-Assisted Test Cases
5. Comparison and Analysis

Overview

The converter application contains two core functions: - `intToRoman(num)` : Converts an integer (1-3999) to Roman numeral representation - `romanToInt(str)` : Converts a valid Roman numeral string to its integer equivalent

Test cases are designed to validate functionality across edge cases, valid inputs, invalid inputs, and boundary conditions.

Function Analysis

Function 1: `intToRoman(num)`

Purpose: Convert integer to Roman numeral string **Input:** Integer value **Output:** Roman numeral string **Constraints:** Valid range 1-3999, subtractive notation correctly implemented

Function 2: `romanToInt(str)`

Purpose: Convert Roman numeral string to integer **Input:** Roman numeral string **Output:** Integer value **Constraints:** Strict format validation, subtractive notation (IV, IX, XL, XC, CD, CM), case-insensitive handling

---

Set 1: Manual Test Cases (27 total)

Methodology for Manual Test Cases

Testing Approach: 1. **Boundary Analysis**: Minimum (1) and maximum (3999) values 2. **Equivalence Partitioning**: Powers of 10, subtractive cases, base symbols 3. **Error Guessing**: Invalid inputs (0, negatives, out-of-range) 4. **Structural Coverage**: All conversion rules tested 5. **Format Validation**: Correct and malformed patterns

Test Cases for `intToRoman(num)` - 13 cases

TC ID	Input	Expected Output	Category	Rationale
IOR-1	1	"I"	Boundary (Min)	Minimum valid input
IOR-2	3999	"MMMCMXCIX"	Boundary (Max)	Maximum valid input
IOR-3	5	"V"	Base Symbol	V representation
IOR-4	10	"X"	Power of 10	X representation

IOR-5	50	“L”	Base Symbol	L representation
IOR-6	100	“C”	Power of 10	C representation
IOR-7	500	“D”	Base Symbol	D representation
IOR-8	1000	“M”	Power of 10	M representation
IOR-9	4	“IV”	Subtractive	I before V
IOR-10	9	“IX”	Subtractive	I before X
IOR-11	99	“XCIX”	Complex	Multiple subtractive rules
IOR-12	2024	“MMXXIV”	Real-world	Current year
IOR-13	0	Error	Invalid	Zero not allowed

Test Cases for `romanToInt(str)` - 14 cases

TC ID	Input	Expected Output	Category	Rationale
ROI-1	“I”	1	Boundary (Min)	Minimum val input
ROI-2	“MMMCMXCIX”	3999	Boundary (Max)	Maximum valid input

ROI-3	“V”	5	Base Symbol	V representatic
ROI-4	“X”	10	Power of 10	X representatic
ROI-5	“L”	50	Base Symbol	L representatic
ROI-6	“C”	100	Power of 10	C representatic
ROI-7	“D”	500	Base Symbol	D representatic
ROI-8	“M”	1000	Power of 10	M representatic
ROI-9	“IV”	4	Subtractive	I before V
ROI-10	“IX”	9	Subtractive	I before X
ROI-11	“XCIX”	99	Complex	Multiple subtractive rules
ROI-12	“MMXXIV”	2024	Real-world	Current year
ROI-13	“mmxxiv”	2024	Case-insensitive	Lowercase input
ROI-14	“IIII”	Error	Invalid Format	Violates repetition rules

Set 2: AI-Assisted Test Cases (26 total)

Prompts Used for AI Generation

Prompt 1 - intToRoman():

```
You are a QA engineer. Generate comprehensive test cases
for intToRoman() covering:
- Boundary values (min 1, max 3999)
- All powers of 10 (1, 10, 100, 1000)
- All base symbols (V, L, D, M)
- Subtractive notations (4, 9, 40, 90, 400, 900)
- Complex combinations and invalid inputs
```

Prompt 2 - romanToInt():

```
You are a QA engineer. Generate comprehensive test cases
for romanToInt() covering:
- Valid individual symbols and combinations
- All subtractive patterns (IV, IX, XL, XC, CD, CM)
- Boundary cases and invalid format violations
- Case insensitivity validation
```

AI-Generated Test Cases for `intToRoman()` - 13 cases

TC ID	Input	Expected Output	Reasoning
AIT1	1	"I"	Boundary validation
AIT2	3999	"MMMCMXCIX"	Comprehensive rule validation
AIT3	5	"V"	Base symbol
AIT4	10	"X"	Power of 10
AIT5	50	"L"	Base symbol
AIT6	100	"C"	Power of 10
AIT7	500	"D"	Base symbol

AIT8	1000	“M”	Power of 10
AIT9	4	“IV”	Subtractive I-V
AIT10	9	“IX”	Subtractive I-X
AIT11	40	“XL”	Subtractive X-L
AIT12	90	“XC”	Subtractive X-C
AIT13	400	“CD”	Subtractive C-D

AI-Generated Test Cases for `romanToInt()` - 13 cases

TC ID	Input	Expected Output	Reasoning
AIT14	“I”	1	Boundary validation
AIT15	“MMMCMXCIX”	3999	Comprehensive validation
AIT16	“V”	5	Base symbol
AIT17	“X”	10	Power of 10
AIT18	“L”	50	Base symbol
AIT19	“C”	100	Power of 10
AIT20	“D”	500	Base symbol
AIT21	“M”	1000	Power of 10
AIT22	“IV”	4	Subtractive I-V
AIT23	“IX”	9	Subtractive I-X
AIT24	“XL”	40	Subtractive X-L
AIT25	“CD”	400	Subtractive C-D
AIT26	“mmxxiv”	2024	Case insensitivity

Comparison and Analysis

Quantitative Comparison

Metric	Manual (Set 1)	AI-Assisted (Set 2)	Total
intToRoman cases	13	13	26
romanToInt cases	14	13	27
Total Cases	**27**	**26**	**53**
Valid cases	24	26	50
Error cases	3	0	3

Key Differences

1. Test Selection Strategy

- **Set 1**: Risk-based approach combining multiple rules per case for efficiency
- **Set 2**: Systematic approach testing each pattern in isolation for clarity

Analysis: Set 1 is more practical for production regression testing; Set 2 is clearer for comprehensive documentation and audit trails.

2. Error Case Coverage

- **Set 1**: Explicitly includes invalid input testing (0, IIII)
- **Set 2**: Focuses on valid pattern coverage

Analysis: Set 1 demonstrates error handling; Set 2 assumes validation exists.

3. Subtractive Notation Testing

- **Set 1**: Tests via 99 (XC+IX) and individual cases (4, 9)
- **Set 2**: Tests each pattern separately (4, 9, 40, 90, 400) for explicit documentation

Analysis: Both cover all six patterns (IV, IX, XL, XC, CD, CM); Set 1 more efficient, Set 2 more explicit.

4. Methodology Comparison

Set 1 - Manual Methodology (Risk-Based): - Identifies critical boundaries (1, 3999) - Maps algorithmic rules (7 symbols, 6 subtractive patterns) - Applies classic testing techniques - Focuses on high-risk code paths - **Strength:** Efficient test selection targeting critical areas - **Weakness:** May miss edge case combinations

Set 2 - AI-Assisted Methodology (Systematic): - Systematically applies requirements - Generates sequential pattern progressions - Ensures no pattern forgotten - Cross-validates against specification - **Strength:** Thorough pattern documentation and completeness verification - **Weakness:** Potential redundancy in similar test cases

---

Recommendations for Testing Strategy

For Regression Testing (Quick QA Cycles): Use Set 1 - 27 cases cover all patterns with minimal redundancy

For Comprehensive Audits (Full Coverage Verification): Use Set 2 - 26 cases document each pattern explicitly

Optimal Combined Strategy: 1. Use Set 1 as primary test suite (efficient coverage) 2. Use Set 2 for pattern verification (ensures completeness) 3. Maintain error cases from Set 1 for robustness validation

---

Conclusion

This optimized test case comparison demonstrates two complementary testing methodologies:

- **Manual Testing (Set 1)**: Risk-focused, efficient test selection targeting high-impact code paths
- **AI-Assisted Testing (Set 2)**: Systematic, exhaustive pattern documentation ensuring completeness

Coverage Summary

Both sets comprehensively cover conversion function requirements with **53 total test cases**:

✓ All 7 base symbols: I(1), V(5), X(10), L(50), C(100), D(500), M(1000) ✓ All 6 subtractive patterns: IV(4), IX(9), XL(40), XC(90), CD(400), CM(900) ✓ Boundary values: minimum (1) and maximum (3999) ✓ Complex combinations: multiple subtractive rules in single numbers ✓ Error conditions: invalid inputs and format violations ✓ Case sensitivity: lowercase input validation ✓ Real-world examples: year 2024 in both formats

This represents an optimal balance between comprehensive coverage and testing efficiency, suitable for both academic evaluation and production quality assurance.

End of Document